사용 매뉴얼 · 비IT 실무자용

2026-10 · Framework v0.2

IT를 몰라도 앱을 만들고 가꾸는 법

자기 개선 개발 프레임워크 사용 매뉴얼 — Git과 GitHub 기초부터 매일의 사용, 문제 해결까지



이 매뉴얼 읽는 법

1 Git과 GitHub 기초 04–15

2 프레임워크가 해 주는 일 16–20

3 처음 한 번 준비하기 21–27

4 앱 만들기: 7단계 28–36

5 앱을 계속 가꾸기 37–41

6 멈췄을 때 · 부록 42–47

색깔 약속

- 당신이 하는 일
- 자동으로 일어나는 일
- AI가 제안하는 일
- 멈춤 · 주의

화면 그림은 이해를 돕는 단순화 그림이다. 실제 GitHub 화면과 색·배치가 조금 다를 수 있다.

처음이라면 1부부터 순서대로, 이미 GitHub를 써 봤다면 2부부터 읽어도 된다

당신은 ‘무엇을’만 정한다

당신이 하는 일 · 4가지

1 원하는 것을 글로 쓴다

2 AI의 계획을 읽고 승인한다

3 완성된 결과를 확인하고 반영한다

4 AI가 낸 개선 후보를 고르거나 거절한다

맡기는 일

코드 작성, 테스트, 검사, 리뷰, 수정, 결과 정리

AI와 자동 검사(로봇)가 한다. 당신은 코드를 읽을 줄 몰라도 된다.

다만 계획과 결과를 **확인하는 눈**은 필요하다. 이 매뉴얼이 그 눈을 기르는 법을 알려 준다.

‘어떻게’는 AI와 자동 검사가 맡고, 최종 결정은 언제나 사람이 한다

01

PART 01 · 슬라이드 04-15

Git과 GitHub, 처음부터

개발자가 아니어도 알아야 할 것만, 일상의 비유로

‘최종_진짜최종’에서 벗어나기

GIT 없이

- 보고서_최종.docx
- 보고서_최종2.docx
- 보고서_진짜최종.docx
- 보고서_진짜최종(수정).docx

누가, 언제, 무엇을 바꿨는지 알 수 없다

→

GIT으로

보고서.docx ← 파일은 하나

10/14	김OO · 표 추가
10/15	AI · 오타 수정
10/17	김OO · 결론 수정

기록이 남고, 언제든 그 시점으로 돌아갈 수 있다

Git · 변경 기록을 자동으로 남기는 도구

GitHub · 그 기록을 인터넷에 두고 함께 쓰는 곳

앱은 수백 개의 파일로 되어 있다. 파일마다 ‘최종’ 사본을 만들 수 없어서 Git을 쓴다

저장소는 기록이 남는 서류함

my-app

README.md	앱 소개서
src/	앱 프로그램(코드)
test/	자동 검사 문제지
.github/	로봇 업무 설명서

변경 기록 언제 누가 무엇을 바꿨는지 모두 보관

앱 하나 = 저장소 하나

앱의 모든 파일과 모든 변경 기록이 한 곳에 있다.

공개(Public) / 비공개(Private)

누가 볼 수 있는지 정한다. 앱 저장소는 공개, AI 작업실은 비공개로 둔다.

주인이 열쇠를 가진다

저장소를 만든 계정(당신)이 설정과 권한을 관리한다.

GitHub 주소 모양: github.com/계정이름/저장소이름

커밋은 저장 시점의 사진



커밋 = 사진 한 장

어느 순간의 파일 전체 상태를 찍어 둔다. 무엇을 바꿨는지 메모가 함께 붙는다.

SHA = 사진의 지문

사진마다 붙는 고유번호다. 내용이 한 글자만 달라도 번호가 바뀐다. 보통 앞 7자리로 줄여 쓴다.

프레임워크는 지문을 대조한다

검사한 것과 반영할 것이 같은 SHA인지 확인한다. 다르면 멈춘다.

그림 속 번호는 예시다. 실제 SHA는 40자리이고, 화면에는 앞 7자리가 보인다

브랜치는 원본 옆의 작업대

원본(main)은 그대로 두고, 옆 작업대에서 고친 뒤, 확인이 끝나면 합친다.



안심 포인트 AI의 결과는 항상 별도 작업대에 올라간다. main은 당신이 Merge할 때만 바뀐다.

점 하나가 커밋 하나다. 작업대의 커밋은 Merge 전까지 원본에 영향을 주지 않는다

PR은 ‘반영해도 될까요?’ 요청서



1 **제목과 설명** · 무엇을 왜 바꿨나

2 **변경된 파일** · 어느 파일이 어떻게 바뀌었나

3 **검사 결과** · 초록 체크면 통과

4 **Merge 버튼** · 누르면 원본에 반영된다

프레임워크에서는 **AI가 PR을 만들고, 당신이 Merge 버튼을 누른다.**

‘Pull Request’는 “내 작업대 내용을 끌어가(pull) 주세요”라는 요청이라는 뜻이다

Merge는 결재 도장

작업대 · ai-publish/…

검사를 마친 AI의 결과

→

원본 · main

실제로 쓰이는 앱



합쳐도 기록은 남는다

누가 언제 무엇을 합쳤는지 PR 화면에 그대로 남는다.

되돌릴 수도 있다

합친 PR 화면의 Revert 버튼은 ‘되돌리는 새 요청서’를 만든다. 그것도 확인 후 Merge한다.

도장은 사람만

프레임워크에는 자동 Merge가 없다. 마지막 결정은 언제나 당신이다.

Merge(머지)는 ‘합치다’라는 뜻이다

Issue는 요청 카드, 댓글은 대화

Issue #8 **Open**

[업무 요구] 고객 이름으로 주문 찾기

주문이 많아 특정 고객의 주문을 찾는 데 오래 걸린다...

로봇	조언 댓글 · 본문 점검 결과
AI	PLAN 제안 · 바꿀 파일 3개
나	**PLAN-승인**

Issue 하나 = 요청 하나

기능 요구도, 고장 신고도 Issue로 쓴다.

댓글로 대화한다

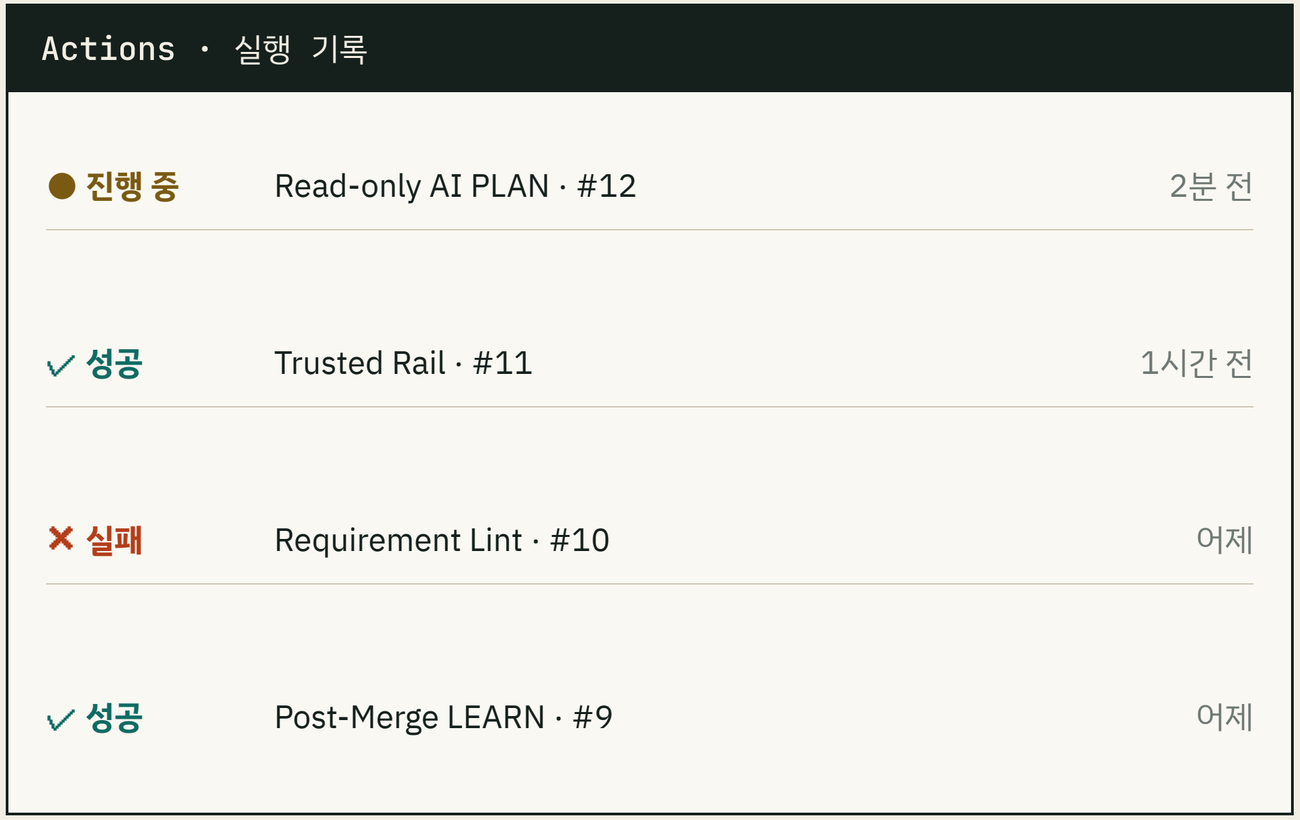
프레임워크는 진행 상황을 그 Issue의 댓글로 알려 준다.

Open / Closed

처리 중이면 열림, 끝났거나 하지 않기로 하면 닫힘.

Issue는 저장소의 Issues 탭에 모인다. 번호(#8)로 서로를 가리킨다

Actions는 규칙대로 일하는 로봇



workflow = 업무 설명서

.github/workflows 폴더의 설명서대로 로봇이 일한다.
프레임워크의 단계 대부분이 여기서 돈다.

색만 보면 된다

진행 중 · 성공 · 실패. 실제 GitHub에서는 노랑 · 초록 · 빨강으로 보인다.

실패해도 원본은 안전

실패하면 그 자리에서 멈출 뿐이다. 이유는 Issue 댓글로 온다.

평소에는 Actions 탭을 볼 일이 거의 없다. 멈췄을 때 원인을 찾는 곳이다

로봇에게 맡기는 열쇠와 메모

SECRET · 금고 속 열쇠

한 번 넣으면 다시 볼 수 없다

비밀번호 같은 값. 로봇만 꺼내 쓴다. 저장한 뒤에는 사람에게도 값이 보이지 않는다.

`EXECUTOR_DISPATCH_TOKEN · ••••••`

VARIABLE · 설정 메모

누구나 볼 수 있는 설정값

비밀이 아닌 값. 예를 들어 ‘AI 작업실이 어디인가’ 같은 주소를 적는다.

`AI_EXECUTOR_REPOSITORY · team/executor`

넣는 곳 저장소 › Settings › Secrets and variables › Actions

값을 Issue·댓글에 붙이지 않는다

실제 이름과 값은 3부 ‘연결 값 등록’에서 다룬다. 값은 설정 도우미가 알려 준다

저장소 화면의 다섯 탭

Code	Issues	Pull requests	Actions	Settings
가끔	자주	자주	멈췄을 때	처음 한 번
파일 보기	**요구 쓰기**	**결과 확인**	**로봇 상태**	**설정**
README와 앱 파일을 본다. 첫 화면이다.	요구를 쓰고, PLAN을 읽고, 승인 댓글을 단다.	AI가 만든 PR을 확인하고 Merge한다.	어느 단계가 돌고, 멈췄는지 본다.	연결 값(Secret·Variable)을 넣는다.

2 당신이 매일 쓰는 탭은 **Issues**와 **Pull requests** 두 개뿐이다

탭은 저장소 이름 바로 아래 한 줄에 있다. 화면이 좁으면 일부가 ⋯ 메뉴 안으로 들어간다

한 장으로 보는 비유

용어	일상의 비유	프레임워크에서는
저장소 Repository	기록이 남는 서류함	앱 하나의 모든 파일과 변경 기록
커밋 Commit	저장 시점의 사진	모든 변경의 단위
SHA	사진의 지문	검사한 것과 반영할 것이 같은지 대조
브랜치 Branch	원본 옆 작업대	AI 결과가 올라가는 곳(ai-publish/...)
Pull Request	반영 요청서	AI가 만드는 결과 보고서(MERGE_READY PR)
Merge	결재 도장	당신만 누른다
Issue	요청 카드	요구를 쓰고 진행 상황을 받는 곳
Actions	규칙대로 일하는 로봇	각 단계를 실행한다

더 많은 용어는 맨 뒤 용어집(46쪽)에 있다

02

PART 02 · 슬라이드 16-20

프레임워크가 해 주는 일

집 짓기에 빗대어 보면 쉽다

집 짓기로 이해하기

당신

건축주

어떤 집이 필요한지 말하고, 설계도에 도장을 찍는다

AI · PLAN

설계사무소

요구를 읽고 설계도(계획)를 제안한다. 직접 짓지는 않는다

AI · 구현

시공팀

설계도 범위 안에서만 짓는다. 현관 열쇠는 받지 않는다

자동 검사

감리단

설계도대로 지었는지, 안전한지 검사하고 기록한다

당신

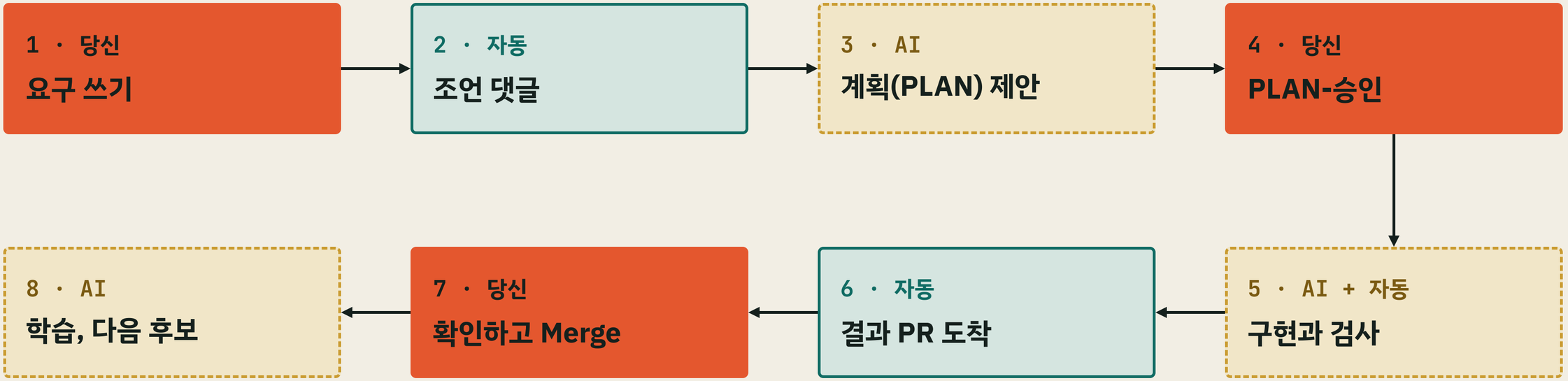
준공 승인

결과를 확인하고 입주를 허락한다. 이것이 Merge다

핵심 규칙 설계도에 도장(PLAN-승인)이 없으면 공사가 시작되지 않는다

감리단이 문제를 찾으면 시공팀이 정해진 횟수만큼 고치고, 그래도 안 되면 공사를 멈추고 건축주를 부른다

요구 하나가 기능이 되는 8단계



1 · 4 · 7 당신의 차례는 세 번뿐이다. 나머지는 기다리면 된다

4부에서 당신 차례인 1·4·7단계와 그 사이에 볼 것을 한 장씩 자세히 설명한다

안심해도 되는 세 가지 약속

1

AI는 원본을 직접 바꿀 수 없다

AI에게는 쓰기 열쇠가 없다. 결과는 언제나 '후보'로 작업대에 올라간다.

2

단계마다 지문을 대조한다

검사한 것과 PR에 올라온 것이 같은 SHA인지 확인한다. 하나라도 다르면 멈춘다.

3

마지막 도장은 사람만 찍는다

자동 Merge는 없다. 계획 승인과 최종 Merge는 당신만 한다.

STOP 수정 한도를 넘거나 무언가 이상하면 자동으로 멈추고 Issue 댓글로 알려 준다. 멈춤은 고장이 아니라 안전장치다.

근거: docs/architecture.md 2장 핵심 원칙 · docs/trust-model.md

알아 둘 저장소 세 곳

공개 · 매일

앱 저장소

당신의 작업장

앱의 파일, 요구(Issue), 결과(PR)가 모두 여기 있다. 당신은 거의 여기서만 일한다.

여기서 하는 일

- 요구 쓰기 · PLAN-승인
- 결과 PR 확인 · Merge
- 개선 후보 고르기

비공개 · 켜 두기

AI 작업실

executor

팀 전용 Claude 구독으로 AI를 부르는 곳. 연결된 컴퓨터가 켜져 있어야 AI 단계가 돈다.

여기서 하는 일

- runner 상태 확인
- 멈췄을 때 기록 보기
- 평소엔 들어갈 일 없음

공개 · 가끔

프레임워크 원본

설명서의 원본

앱 저장소의 프레임워크 파일은 여기서 복사해 온다. 업데이트는 도우미가 맞춘다.

여기서 하는 일

- 매뉴얼·문서 찾아보기
- 업데이트 소식 확인
- 직접 고치지 않음

예: 앱 저장소 erpsarang/transportation_guide · 프레임워크 원본 erpsarang/self-improvement-mvp

03

PART 03 · 슬라이드 21-27

처음 한 번 준비하기

기술 작업은 도우미에게, 확인은 당신이

준비물 네 가지

1 **GitHub 개인 계정**

앱 저장소의 주인이 된다. 지금은 개인 계정 소유 저장소만 지원한다(조직 계정은 아직).

2 **팀 전용 Claude 구독**

AI 비용을 내는 계정이다. 개인 계정으로 쓰면 그 사람의 사용 한도를 함께 쓰게 된다.

3 **켜 둘 컴퓨터 한 대**

AI 작업이 실제로 도는 곳(runner)이다. 일하는 시간에는 켜져 있고 절전에 들지 않아야 한다.

4 **설정 도우미**

개발자 동료나 Claude Code 세션. 처음 한 번 기술 설정을 맡고, 프레임워크 업데이트 때 다시 부른다.

근거: docs/usage.md 2장 AI 실행 경계 준비 · 2-2장 App이 지켜야 할 것

준비 작업 나눠 맡기기

작업	누가	끝났는지 확인하는 법
앱 저장소 만들기	**당신**	github.com/내계정/앱이름 이 열린다
README 쓰기	**당신**	첫 화면에 앱 소개가 보인다
프레임워크 파일 붙이기	**도우미**	Actions 탭에 workflow 목록이 보인다
AI 작업실·컴퓨터 준비	**도우미**	runner 상태가 Idle(대기)이다
연결 값 등록	**당신** + 도우미	Settings에 값 두 개가 보인다
시험 요구로 확인	**당신**	시험 Issue에 PLAN 댓글이 달린다

도우미에게는 이 매뉴얼과 프레임워크 저장소의 docs/introduction.md를 함께 건넨다

준비는 여섯 걸음

1 — 저장소 만들기
당신
New repository, 개인 계정으로

2 — README 쓰기
당신
앱의 목적과 범위

3 — 파일 붙이기
도우미
프레임워크 파일 복사·대조

4 — AI 작업실
도우미
executor, 컴퓨터, Claude 로그인

5 — 연결 값 등록
당신 + 도우미
Settings에 값 두 개

6 — 시험 요구
당신
작은 요구로 PLAN 확인

한 번만 이 준비는 앱마다 한 번이다. 다음부터는 4부의 7단계만 반복한다

3·4단계 기술 절차는 프레임워크 저장소 docs/introduction.md 3부에 있다(도우미용)

README는 앱의 소개서다

AI는 README를 읽고 앱이 무엇을 위한 것인지 이해한다

PLAN이 참고한다

요구를 앱의 목적에 맞게 해석한다.

개선 후보의 기준이 된다

AI가 다음 개선 후보를 고를 때 README의 방향을 따른다.

README.md · 예시

이동 경로 추천 도구

누가 쓰나 · 처음 가는 곳을 찾아가는 나

넣는 것과 얻는 것 · 목적지 주소 → 수단별 경로

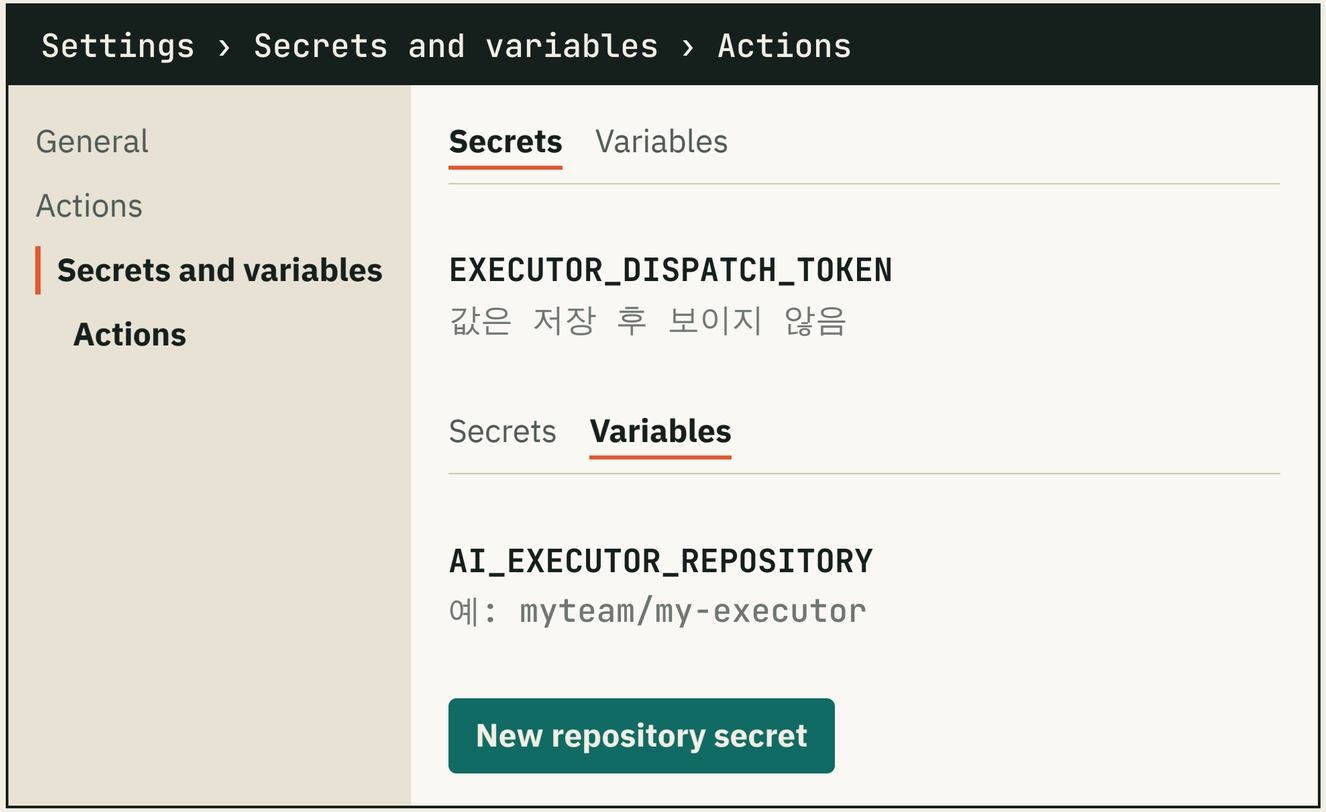
이번 단계 범위 · 도보·자전거·자동차만

지킬 것 · 외부 패키지 없이, 테스트는 가짜 서비스로

검사 명령 · npm test, npm run build

예시는 erpsarang/transportation_guide의 README를 줄인 것이다. 검사 명령 줄은 도우미가 채운다

앱과 AI 작업실을 잇는 두 값



1 저장소의 **Settings** 탭

2 왼쪽 메뉴 **Secrets and variables › Actions**

3 Secrets 탭에서 **New repository secret**, Variables 탭에서 **New repository variable**

4 이름은 글자 그대로, 값은 도우미가 준 것을 붙여 넣고 저장

이름이 한 글자라도 틀리면 AI 단계가 시작 전에 멈춘다

선택 설정(결과 PR 자동 검사용 앱, 개선 후보 자동 시작 등)은 도우미와 상의해 나중에 켠다

네 가지가 보이면 준비 끝

☐ 요구 템플릿이 보인다

`Issues › New issue`

‘업무 요구 → AI PLAN’ 템플릿이 목록에 있다.

☐ 로봇 목록이 보인다

`Actions` 탭 왼쪽

Read-only AI PLAN, Trusted Rail 같은 이름이 줄지어 있다.

☐ AI 컴퓨터가 대기 중이다

`AI 작업실 › Settings › Actions › Runners`

상태가 Idle이면 정상, Offline이면 꺼져 있다.

☐ 시험 요구에 PLAN이 달린다

작은 `[업무 요구]` 하나

몇 분 안에 조언 댓글과 PLAN 댓글이 온다. 승인은 하지 않아도 된다.

시험 요구는 확인 뒤 ‘하지 않음’으로 닫아도 된다(방법은 39쪽)

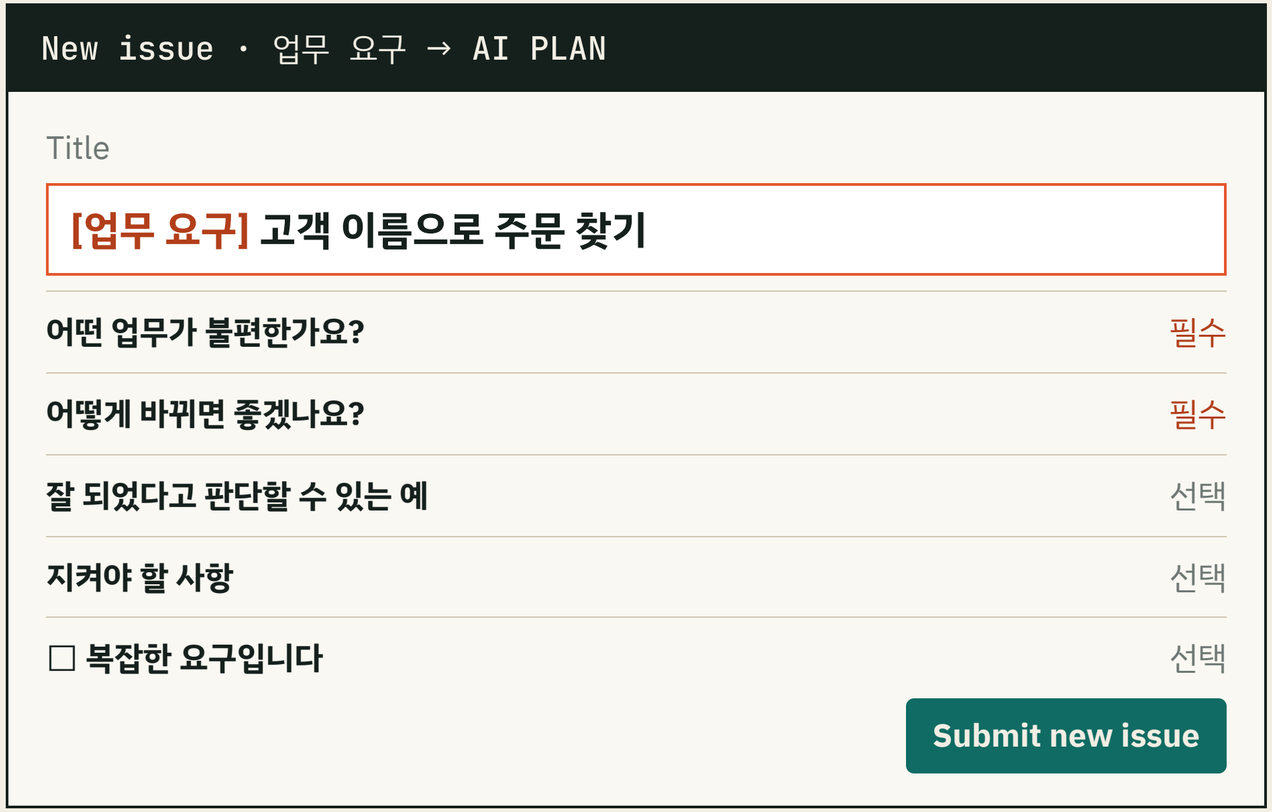
04

PART 04 · 슬라이드 28-36

요구 하나가 기능이 되기까지

당신의 차례는 세 번이다. 쓰고, 승인하고, Merge한다

템플릿에 원하는 결과를 쓴다



1. 앱 저장소의 **Issues** 탭 › **New issue**
2. **업무 요구 → AI PLAN** 템플릿을 고른다
3. 제목의 **[업무 요구]** 뒤에 한 줄 요약
4. 빈칸을 평소 말로 채운다. 회색 예시 줄은 지워도 된다
5. **Submit new issue**

저장소 구성원 계정으로 써야 AI 계획이 자동으로 시작된다

‘복잡한 요구’ 체크는 구조를 크게 바꾸는 요구에만. 체크하면 더 강한 AI 모델이 계획을 짠다

좋은 요구는 작고, 결과가 보인다

아쉬운 요구

[업무 요구] 앱을 더 좋게

검색도 빠르게 하고, 화면도 예쁘게 바꾸고, 엑셀 내보내기도 넣어 주세요.

변경이 셋이다 → Issue 세 개로 나눈다

‘좋게’, ‘예쁘게’는 잘 됐는지 판단할 수 없다

좋은 요구

[업무 요구] 고객 이름으로 주문 찾기

불편한 점 · 주문이 많아 특정 고객의 주문을 찾는 데 오래 걸린다.

바뀌면 · 고객 이름을 넣으면 그 고객의 주문만 보인다.

잘 된 예 · ‘김민수’로 찾으면 다른 고객의 주문은 보이지 않는다.

지킬 것 · 지금 쓰는 날짜 검색도 그대로 된다.

1 요구 하나에 변경 하나

2 잘 된 모습을 예로 든다

3 파일 이름은 몰라도 된다

4 짧게, 한 화면 안에서

본문이 너무 길면 Merge 뒤 학습 단계(LEARN)가 실패한다. 다음 쪽의 조언 댓글이 길이를 재 준다

몇 초 뒤, 로봇의 조언 댓글

`github-actions · 댓글`

요구 본문 점검 (조언)

고칠 점이 없습니다.

R1	통과	LEARN 증거 1,180바이트 / 한도 8,192바이트
R2	정보	PLAN 모델: sonnet (복잡한 요구 체크 없음)
R3	정보	'예상 변경 범위' 절을 찾지 못했습니다 …
R4	정보	… 경로 모양은 판단하지 않았습니다

결정적 규칙 R1~R4 · AI 호출 없음

R1 본문 길이
'오류'면 본문을 줄인다

R2 어느 AI가 계획하나
복잡한 요구 체크에 따라 정해진다

R3 바꿀 파일 수
한 번에 8개까지. 넘으면 요구를 나눈다

R4 헷갈리는 표기
주소·파일 경로는 말로 풀어 쓴다

조언일 뿐 계획을 막지 않는다. '정보'는 넘어가고, '오류'·'경고'가 있으면 본문을 고친다 → 계획이 새 본문으로 다시 시작된다

AI의 계획(PLAN)을 읽는다

AI · 댓글 (예시)

PLAN (AI 제안 — 구현 승인 아님)

준비 상태 `ready=true` Blocking Question 없음

AI가 이해한 목표

고객 이름으로 주문을 거르는 검색을 더한다

변경 예상 파일

`src/orders/search.ts · OrderList.tsx · search.test.ts`

완료조건

‘김민수’로 찾으면 그 고객의 주문만 보인다

범위 밖 / 금지 변경

날짜 검색 동작은 바꾸지 않는다

SHA · artifact 같은 기술 줄은 건너뛴다

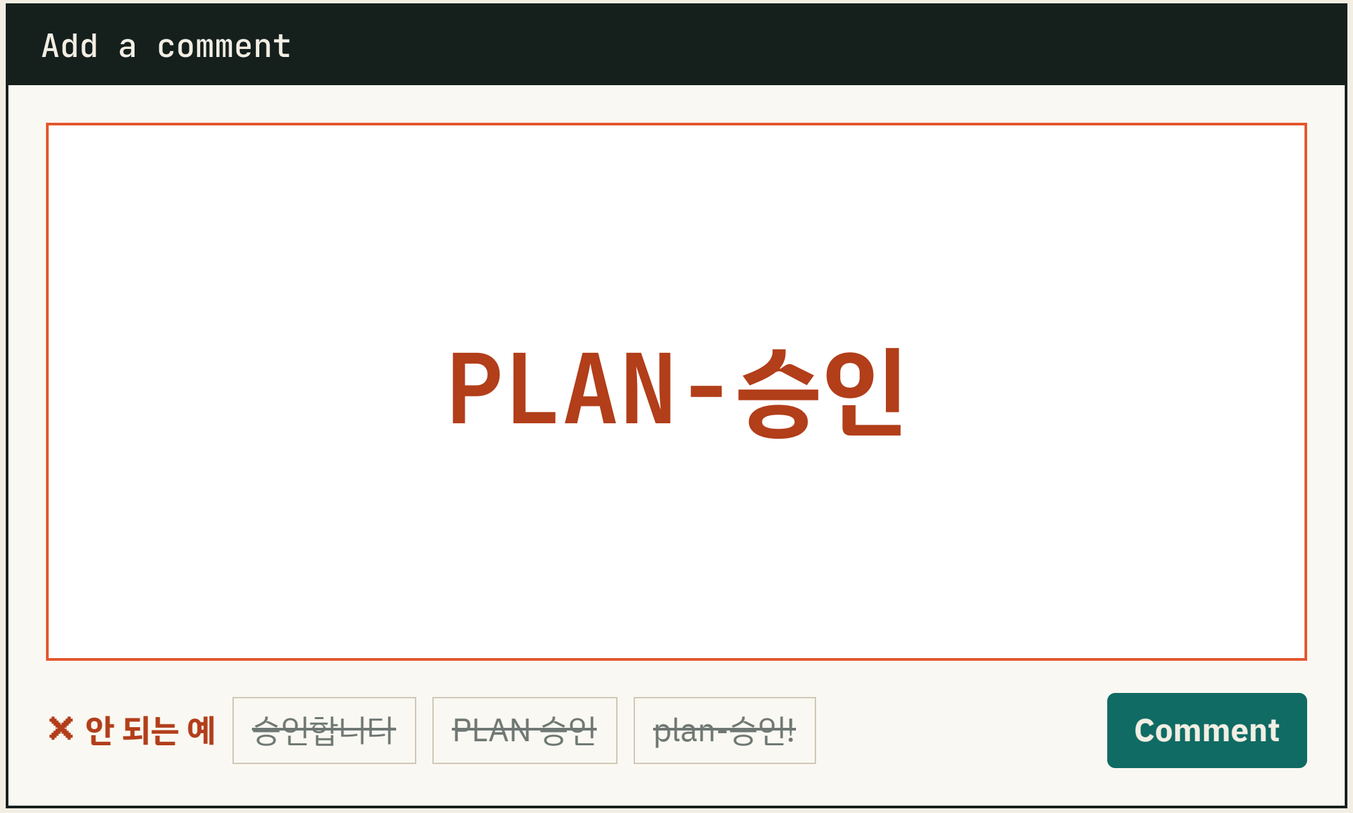
ready=false면 승인이 막혀 있다. 댓글의 Blocking Question에 답하도록 본문을 보완한다

이 네 가지만 묻는다

1. 목표를 내 뜻대로 이해했나
2. 완료조건이 내 ‘잘 된 예’와 같나
3. 지킬 것이 ‘범위 밖’에 들어 있나
4. 준비 상태가 ready=true인가

하나라도 아니면 Issue 본문을 고친다. 새 PLAN이 온다

정확히 ‘PLAN-승인’ 이라고 댓글을 단다



글자 그대로

하이픈 하나, 띄어쓰기 없음. 다른 말을 덧붙이지 않는다.

그 뒤는 자동

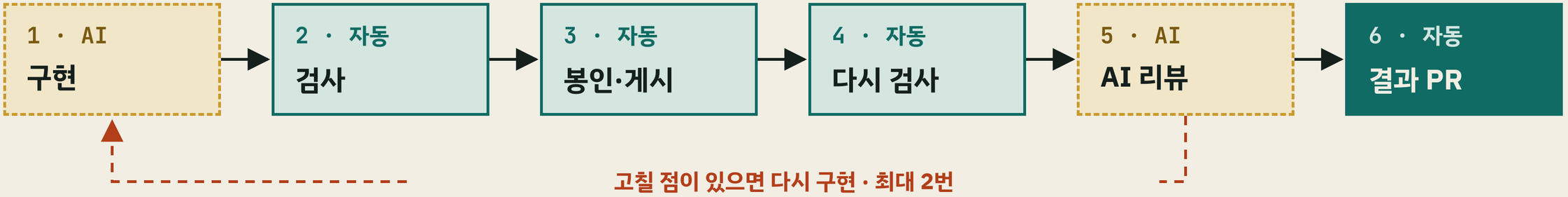
로봇이 승인을 기록하고 구현을 시작한다.

승인 뒤 본문을 고치면

승인이 무효가 된다. 새 PLAN을 읽고 다시 승인한다.

진행하지 않으려면 사유를 댓글로 남기고 Issue를 닫는다(Close issue)

승인 뒤에는 로봇과 AI가 일한다



Issue 댓글로 본다

단계마다 ‘HumanStatus’ 댓글이 지금 상황과 다음 행동을 알려 준다.

Actions 탭 색으로 본다

진행 중 성공 실패

빨강이면 Issue의 STOPPED 댓글부터 읽는다.

AI 컴퓨터는 켜 둔다

꺼져 있으면 단계가 시간 초과로 멈춘다(42–44쪽).

당신이 할 일은 없다. 보통 수십 분이 걸린다. 끝나면 Issue에 결과 PR 링크가 달린다

결과 PR을 네 가지로 확인한다

Pull request #12 **Open**

고객 이름으로 주문 찾기 (#8)

Conversation Commits Checks Files changed 3

1. ✓ **All checks have passed**
2. Files changed · search.ts, OrderList.tsx, search.test.ts
3. 병합 전 확인 · ☐ 이름 입력창이 목록 위에 보인다
4. 설정·비밀 값·배포 파일 변경 없음

Merge pull request

1. **검사가 모두 초록**
   빨간 ✕가 있으면 Merge하지 않는다
2. **파일이 PLAN 목록 안**
   Files changed와 승인한 PLAN을 대조
3. **설명과 '병합 전 확인'**
   요구와 맞는지 읽고, 목록이 있으면 직접 해 본다
4. **운영·보안 영향 없음**
   모르겠으면 도우미에게 묻는다

노란 'Approve and run workflows' 띠가 보이면 한 번 눌러 검사를 실행시킨다

확인했으면 Merge, 이것은 사람만 한다



앱에 반영된다

바뀐 내용이 앱의 기준 원본(main)에 합쳐진다.

요구 Issue가 정리된다

다 끝났으면 닫힌다. 남은 범위가 있으면 열린 채 후속 PLAN이 온다.

AI가 돌아본다

학습(LEARN) 결과가 그 Issue에 댓글로 남는다.

사람만 Merge 버튼은 확인한 사람이 누른다. 자동 Merge는 켜지 않는다

Merge 전에는 요구 Issue를 닫지 않는다. Merge하지 않으려면 PR에 사유를 남기고 Close pull request

05

PART 05 · 슬라이드 37-41

앱을 계속 가꾸기

고장 신고, 개선 후보, 되돌리기, 그리고 일의 리듬

고장도 같은 템플릿, 같은 7단계로

New issue · 업무 요구 → AI PLAN

[업무 요구] 띄어 쓴 이름으로 찾으면 결과가 없음

어떤 업무가 불편한가요?
주문 검색에 ‘김 민수’처럼 띄어 쓰면 아무 주문도 나오지 않는다. ‘김민수’는 된다.

어떻게 바뀌면 좋겠나요?
띄어쓰기와 상관없이 같은 고객의 주문이 나왔으면 한다.

잘 되었다고 판단할 수 있는 예
‘김 민수’와 ‘김민수’의 결과가 같다.

지켜야 할 사항
날짜 검색과 지금의 이름 검색은 그대로 된다.

고장 신고의 세 재료

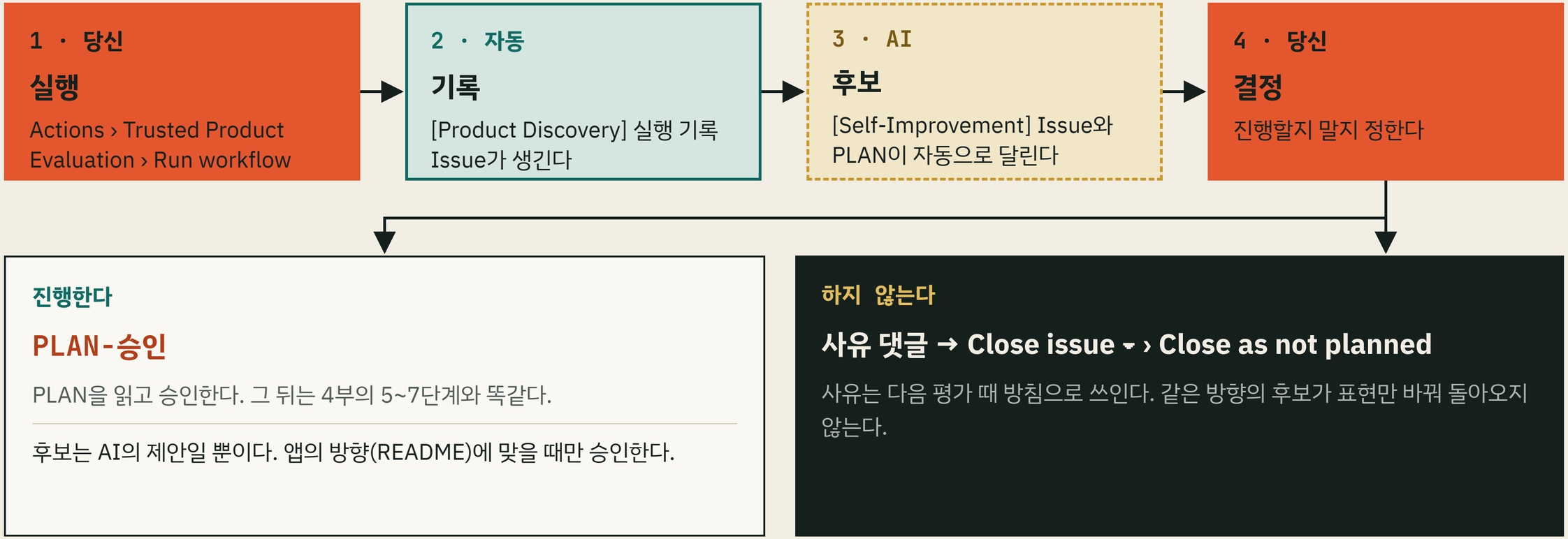
1 어떻게 하면 생기나
누른 순서, 넣은 값

2 무엇을 기대했나
원래 되어야 할 모습

3 실제로 무엇이 됐나
화면 문구를 그대로 옮겨 적는다

AI는 본문의 글을 읽는다. 캡처만 붙이지 말고 화면에 보인 문구를 글로 적는다

AI에게 다음 개선 후보를 묻는다



Run workflow는 기본 브랜치(main)에서 실행한다. 자동 시작은 저장소 설정으로 켤 수 있다(도우미)

마음이 바뀌었을 때

승인 전

이렇게 한다

Issue 본문을 고친다

그러면

PLAN이 새 본문으로 다시 시작된다. Issue 하나에 서로 다른 본문 4개까지 자동

승인 뒤 · Merge 전

이렇게 한다

본문을 고치거나 결과 PR을 닫는다

그러면

이전 승인은 무효. 새 PLAN을 읽고 다시 PLAN-승인

Merge 뒤

이렇게 한다

되돌리는 새 요구를 쓴다

그러면

예: ‘이름 검색을 이전 동작으로 되돌려 주세요’. 급하면 PR의 Revert 버튼(도우미와)

언제든 그만두려면 사유를 댓글로 남기고 Issue를 닫는다

본문 4개 한도를 넘으면 Actions › Read-only AI PLAN을 Issue 번호로 직접 실행한다

매일 · 매주 · 매달

매일 5분

- 알림에서 PLAN·결과 PR이 왔는지 본다
- 멈춤(STOPPED) 댓글이 있는지 본다
- AI 컴퓨터가 켜져 있는지 본다

매주 30분

- Merge된 기능을 앱에서 직접 써 본다
- 오래 열린 요구는 정리해 닫는다
- 개선 후보를 한 번 받아 본다

매달 도우미와

- 프레임워크 업데이트를 맞춘다
- Claude 구독 사용량을 확인한다
- README가 앱의 방향과 맞는지 본다

작게, 자주 큰 요구 하나보다 작은 요구 여럿이 빠르고 안전하다. 하나가 Merge되면 다음을 쓴다

GitHub 알림(Watch)을 켜 두면 새 댓글을 메일과 앱으로 받는다

06

PART 06 · 슬라이드 42-47

막혔을 때와 지킬 것

당황하지 말고 댓글부터 읽는다. 댓글이 다음 행동을 알려 준다

이런 댓글이 보이면

보이는 것	뜻	할 일
`PLAN_FAILED`	AI가 계획을 만들지 못했다	**이유를 읽고 본문을 고친다 → 자동 재시작**
`PLAN 자동 재시도 (1/2)`	일시적 실패, 로봇이 다시 시도 중	**1~2분 기다린다**
`STOPPED · 시간 초과`	AI 단계가 제한 시간 안에 못 끝났다	**AI 컴퓨터부터 확인(다음 쪽)**
`STOPPED · …_FAILED`	AI 작업실이 실패를 알렸다	**Issue 번호를 도우미에게 알린다**
`STRUCTURAL · 수정 소진`	계획대로는 고칠 수 없다고 판단	**요구를 더 작게 나눠 새로 쓴다**
`본문 4개 한도`	본문을 너무 여러 번 바꿨다	**Actions에서 PLAN을 직접 실행**
`Approve and run workflows`	GitHub가 실행 허락을 기다린다	**한 번 누른다**

모든 상태 댓글에는 ‘현재 상황’과 ‘다음 행동’이 적혀 있다. 그 줄부터 읽는다

멈췄다면 먼저 AI 컴퓨터를 본다

AI 작업실 › Settings › Actions › Runners

Runners

● office-pc · self-hosted — Offline

Idle	대기 중 · 정상
Active	일하는 중 · 정상
Offline	꺼짐 · AI 단계가 멈춘다

1. AI 컴퓨터를 켜고 **절전 모드**를 끈다
2. 몇 분 뒤 상태가 **Idle**로 바뀌는지 본다
3. 멈춘 단계를 다시 시작한다. PLAN이면 본문을 고치거나 Actions에서 직접 실행
4. 구현·리뷰 단계였다면 요구 Issue 번호를 도우미에게 알린다

AI 작업실 Actions에 Queued 실행이 쌓였다면 컴퓨터가 꺼진 것이다

AI 단계는 결과를 약 15분(PLAN은 약 10분) 기다린다. 그 안에 컴퓨터가 일을 받지 못하면 멈춘다

이것만은 하지 않는다

✕ 비밀 값 붙여 넣기

토큰·비밀번호는 Issue·댓글·채팅에 쓰지 않는다.

대신 → Settings › Secrets에만 넣는다

✕ 확인 없이 Merge

검사 결과와 바뀐 파일을 보기 전에는 누르지 않는다.

대신 → 35쪽의 네 가지를 본다

✕ 프레임워크 파일 수정

.github/, src/self-improvement/, policy/는 안전장치다.

대신 → 도우미에게 맡긴다

✕ 앱 코드 직접 고치기

직접 고치면 검사와 기록이 남지 않는다.

대신 → 요구로 쓴다

✕ 낯선 사람의 승인

공개 저장소에서는 모르는 계정의 PLAN-승인도 구현을 시작시킨다.

대신 → 결과 PR을 닫고 도우미에게 알린다

✕ 요구 하나에 여러 변경

한 번에 바꿀 수 있는 파일은 8개까지다.

대신 → Issue를 나눠 쓴다

실수했다면 숨기지 말고 도우미에게 바로 알린다. 비밀 값이 노출됐다면 즉시 새 값으로 바꾼다

용어 한눈에

저장소	Repository	앱 파일과 기록을 담는 폴더
커밋	Commit	변경을 저장한 한 장의 스냅숏
브랜치	Branch	원본을 건드리지 않는 작업용 사본
PR	Pull Request	브랜치를 원본에 합쳐 달라는 요청
Merge	머지	PR을 원본에 합치기. 사람만 한다
Issue	이슈	요구나 고장을 적는 카드
Actions	액션	저장소에서 도는 자동 로봇
Secret	비밀 값	저장 뒤에는 보이지 않는 설정 값

Runner	러너	Actions 일을 실제로 하는 컴퓨터
Executor	AI 작업실	팀 Claude 구독으로 AI를 부르는 비공개 저장소
PLAN	계획	AI가 제안한 구현 계획
PLAN-승인	승인 댓글	계획대로 구현하라는 사람의 신호
Trusted Rail	검증 레일	결과를 봉인하고 다시 검사하는 자동 구간
REVIEW	AI 리뷰	결과가 계획과 맞는지 보는 AI 심사
LEARN	학습	Merge 뒤 이번 일을 돌아보는 단계
Product Discovery	개선 탐색	다음 개선 후보를 찾는 평가

왼쪽 여덟 개는 GitHub의 말, 오른쪽 여덟 개는 이 프레임워크의 말이다

이 한 장만 기억하면 된다

요구 하나의 일곱 단계

1	요구 쓰기	당신
2	조언 댓글 확인	자동
3	PLAN 읽기	당신
4	PLAN-승인 댓글	당신
5	기다리기	AI + 자동
6	결과 PR 확인	당신
7	Merge	당신

막히면 ① 댓글의 '다음 행동'을 읽고 ② AI 컴퓨터를 확인하고 ③ 도우미에게 Issue 번호를 알린다

세 가지 약속

요구 하나에 변경 하나

작게 나눌수록 빠르고 안전하다

비밀 값은 Settings에만

Issue·댓글·채팅에는 쓰지 않는다

Merge는 확인한 사람이

네 가지를 보고 나서 누른다